APPLICATION DEVELOPMENT ASSIGNMENT

PROJECT 2 | WEB APPLICATION

Student Name	Abeera Iman
Registration ID	FA26-BSCS-0175
Section	DM
Project Title	Student Task Manager
Application Type	Web Application

Submitted for: Application Development

Table of Contents

1. Introduction and Objective
2. Project Overview
3. Features and Requirements
4. Tools and Technologies
5. Application Screenshots
6. Source Code
7. Testing and Results
8. GitHub Publishing Instructions
9. Conclusion

1. Introduction and Objective

The objective of this assignment is to apply self-learning and modern software development practices by designing and implementing a functional application. For this project, a responsive web application named Student Task Manager has been developed. It helps students organize study activities, monitor completion status, and manage priorities through a simple interface.

2. Project Overview

Student Task Manager is a browser-based application created with HTML, CSS, and JavaScript. Users can enter study tasks, assign a priority, mark work as completed, filter tasks, and remove tasks that are no longer needed. Task information is stored in the browser using localStorage, so the list remains available after refreshing the page on the same browser.

3. Features and Requirements

Feature	Description
Add task	Enter a task description and add it to the list.
Priority selection	Choose High, Normal, or Low priority for each task.
Completion status	Use the checkbox to mark a task completed or pending.
Task filters	View all tasks, pending tasks, or completed tasks.
Delete task	Remove an individual task from the list.
Progress counters	Display total, completed, and pending task counts.
Local storage	Save tasks in the current browser automatically.
Responsive layout	Adapt the interface for desktop and mobile screen widths.

4. Tools and Technologies

- HTML5: structures the web page and form elements.

- CSS3: provides colors, spacing, responsive layout, and visual styling.
- JavaScript: handles task creation, completion, filtering, deletion, and counters.
- Browser localStorage: stores the task list locally in the user's browser.
- Visual Studio Code or any text editor and a modern web browser can be used to edit and run the project.

5. Application Screenshots

The following screenshots were captured from the running application during functional testing. They show the main dashboard and the key operations required by the project.

Figure 1. Main dashboard

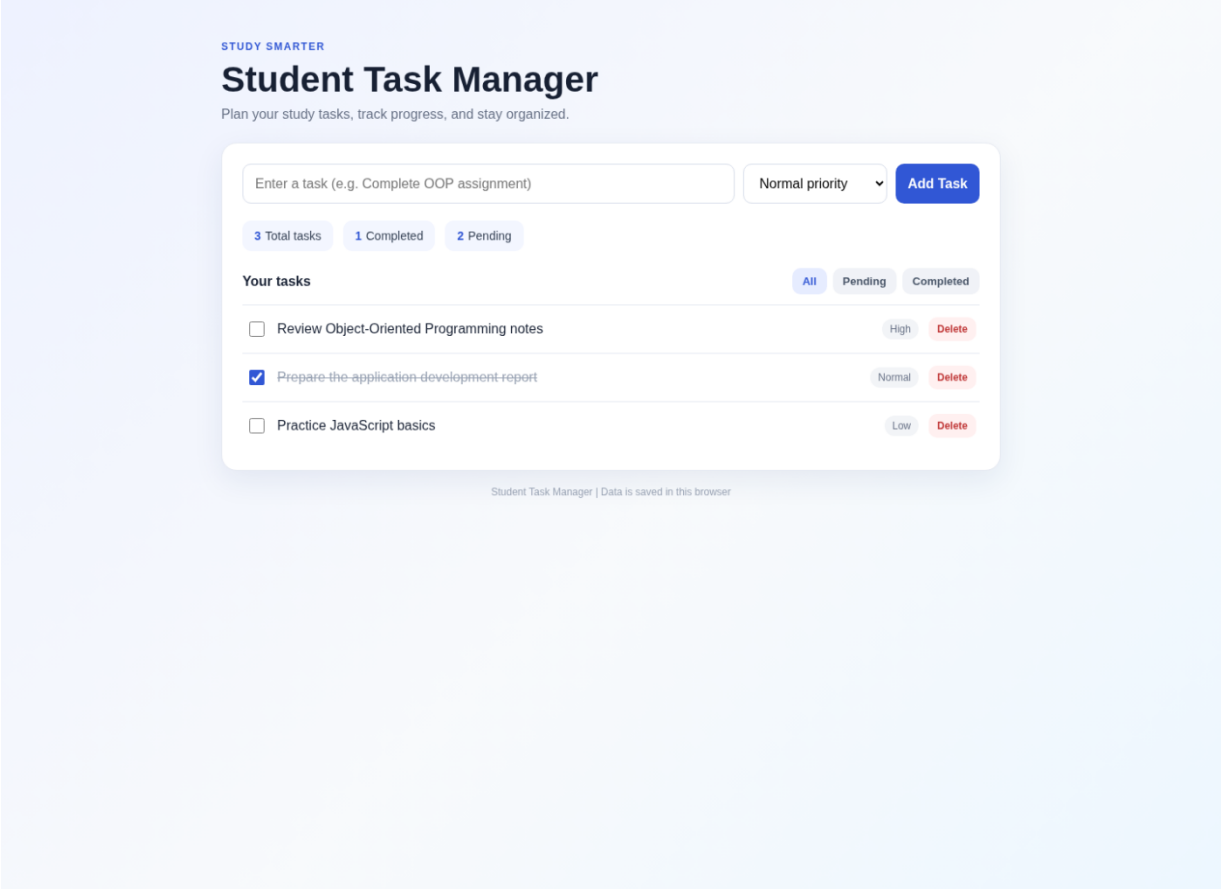


Figure 2. Adding a new task

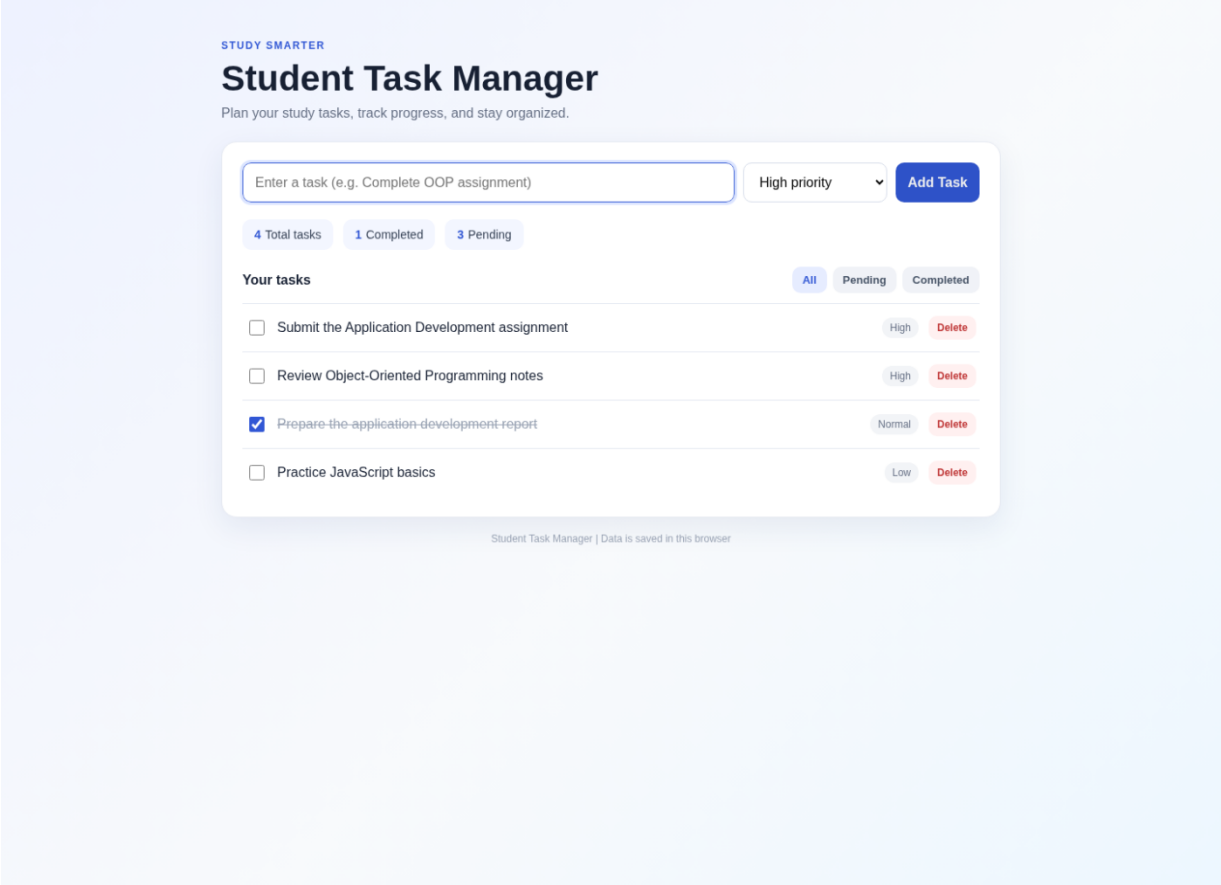


Figure 3. Completed-task filter

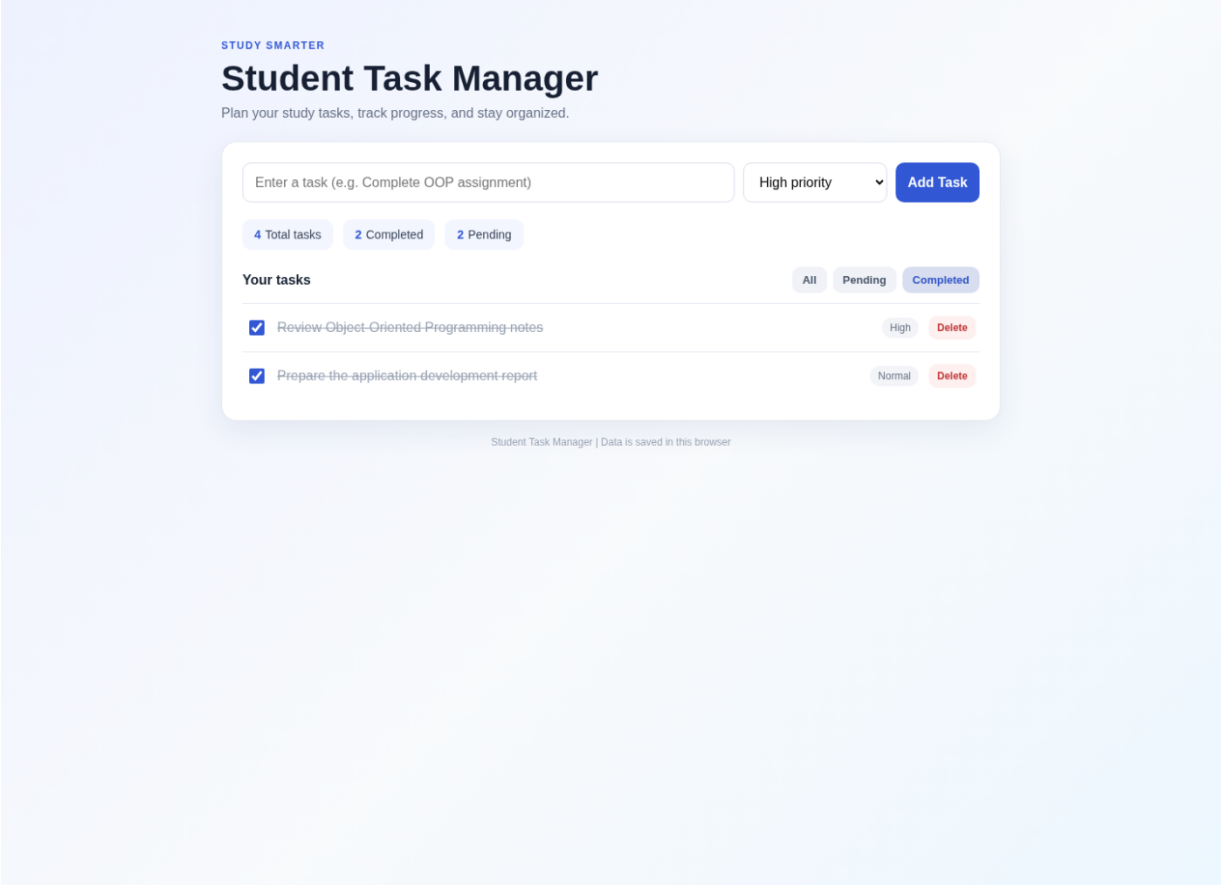
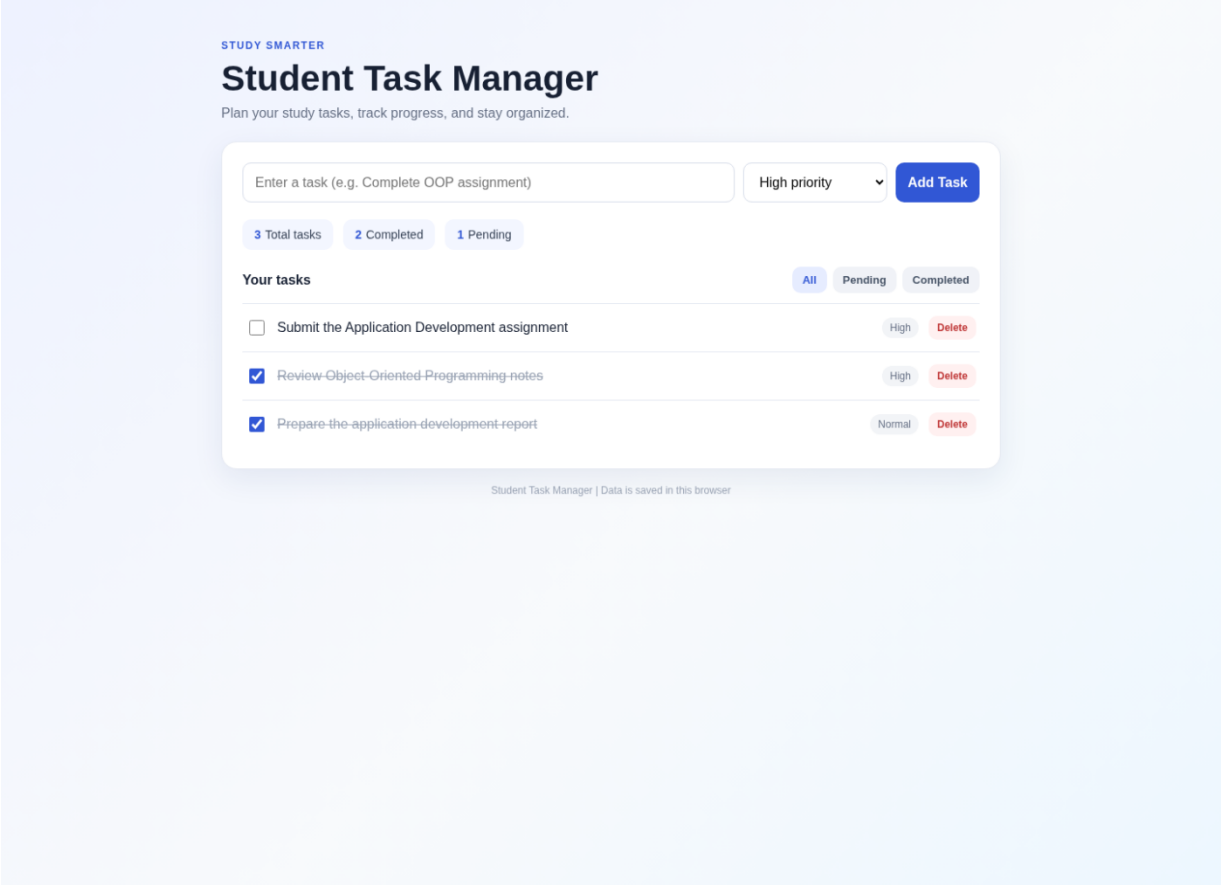


Figure 4. Deleting a task



6. Source Code

The complete runnable source code is included below. Save it as index.html and open the file in a modern browser. No external packages or server are required.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta name="description" content="A simple student task manager to organize study tasks." />
  <title>Student Task Manager</title>
  <style>
    :root { --ink:#172033; --muted:#667085; --blue:#3157d5; --pale:#f3f6ff; --line:#e5e9f2; }
    * { box-sizing:border-box; }
    body { margin:0; min-height:100vh; font-family:Arial,Helvetica,sans-serif; color:var(--ink); background:linear-
gradient(135deg,#eef2ff,#f8fafc 55%,#edf7ff); }
    .wrap { width:min(920px,92%); margin:48px auto; }
    header { margin-bottom:24px; }
    .tag { color:var(--blue); font-weight:700; font-size:12px; letter-spacing:1.5px; text-transform:uppercase; }
    h1 { margin:8px 0; font-size:clamp(28px,5vw,42px); }
    .sub { margin:0; color:var(--muted); }
    .panel { background:#fff; border:1px solid var(--line); border-radius:18px; padding:24px; box-shadow:0 12px 35px
rgba(31,52,110,.08); }
    form { display:grid; grid-template-columns:1fr 170px auto; gap:10px; }
    input,select,button { font:inherit; border-radius:10px; padding:13px 14px; }
    input,select { border:1px solid #d8deeb; min-width:0; background:#fff; }
    input:focus,select:focus { outline:3px solid #dce5ff; border-color:var(--blue); }
    button { border:0; background:var(--blue); color:#fff; font-weight:700; cursor:pointer; }
    button:hover { filter:brightness(.94); }
    .stats { display:flex; gap:12px; margin:20px 0; flex-wrap:wrap; }
    .stat { background:var(--pale); border-radius:11px; padding:10px 14px; color:#344054; font-size:14px; }
    .stat strong { color:var(--blue); margin-right:5px; }
    .toolbar { display:flex; justify-content:space-between; align-items:center; gap:12px; margin:8px 0 12px; }
    .filters { display:flex; gap:7px; flex-wrap:wrap; }
    .filters button { padding:8px 12px; color:#475467; background:#f0f2f7; font-size:13px; }
    .filters button.active { background:#e6ecff; color:var(--blue); }
```

```
    #taskList { list-style:none; margin:0; padding:0; }
    .task { display:flex; align-items:center; gap:12px; padding:14px 4px; border-top:1px solid var(--line); }
    .task input { width:18px; height:18px; accent-color:var(--blue); }
    .task-text { flex:1; overflow-wrap:anywhere; }
    .task.done .task-text { color:#98a2b3; text-decoration:line-through; }
    .priority { color:#667085; font-size:12px; background:#f2f4f7; border-radius:20px; padding:5px 9px; }
    .delete { background:#fff0f0; color:#c03636; padding:7px 10px; font-size:12px; }
    .empty { text-align:center; color:var(--muted); padding:28px 8px; }
    footer { text-align:center; color:#98a2b3; font-size:12px; margin:18px 0; }
    @media(max-width:650px) { .wrap{margin:28px auto} .panel{padding:17px} form{grid-template-columns:1fr} .toolbar{align-
items:flex-start;flex-direction:column} }
  </style>
</head>
<body>
  <main class="wrap">
    <header>
      <div class="tag">Study smarter</div>
      <h1>Student Task Manager</h1>
      <p class="sub">Plan your study tasks, track progress, and stay organized.</p>
    </header>
    <section class="panel" aria-label="Task manager">
      <form id="taskForm">
        <input id="taskInput" type="text" maxlength="100" placeholder="Enter a task (e.g. Complete OOP assignment)" aria-
label="Task description" required />
        <select id="priorityInput" aria-label="Task priority"><option value="Normal">Normal priority</option><option
value="High">High priority</option><option value="Low">Low priority</option></select>
        <button type="submit">Add Task</button>
      </form>
      <div class="stats" aria-live="polite">
        <div class="stat"><strong id="totalCount">0</strong>Total tasks</div>
        <div class="stat"><strong id="doneCount">0</strong>Completed</div>
        <div class="stat"><strong id="pendingCount">0</strong>Pending</div>
      </div>
      <div class="toolbar">
        <strong>Your tasks</strong>
        <div class="filters" aria-label="Task filters">
          <button type="button" class="active" data-filter="all">All</button>
          <button type="button" data-filter="pending">Pending</button>
          <button type="button" data-filter="done">Completed</button>
        </div>
      </div>
      <ul id="taskList"></ul>
      <div id="emptyState" class="empty">No tasks yet. Add your first study task above.</div>
    </section>
    <footer>Student Task Manager | Data is saved in this browser</footer>
  </main>
  <script>
    // Keep the selected filter and task records in memory/browser storage.
    const STORAGE_KEY = 'studentTaskManager.tasks.v1';
    let tasks = loadTasks();
    let currentFilter = 'all';
    // A few editable sample entries make the first run easy to explore.
    if (tasks.length === 0) {
      tasks = [
        { id: 'demo-1', text: 'Review Object-Oriented Programming notes', priority: 'High', done: false },
        { id: 'demo-2', text: 'Prepare the application development report', priority: 'Normal', done: true },
        { id: 'demo-3', text: 'Practice JavaScript basics', priority: 'Low', done: false }
      ];
      saveTasks();
    }

    function loadTasks() {
      try {
        const saved = JSON.parse(localStorage.getItem(STORAGE_KEY) || '[]');
        return Array.isArray(saved) ? saved : [];
      } catch (error) {
        console.warn('Saved tasks could not be read:', error);
        return [];
      }
    }

    function saveTasks() {
      try {
        localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
      } catch (error) {
        alert('Your browser could not save the task list. Please check browser storage settings.');
      }
    }

    function updateStats() {
      const completed = tasks.filter(task => task.done).length;
      document.getElementById('totalCount').textContent = tasks.length;
```

```
      document.getElementById('doneCount').textContent = completed;
      document.getElementById('pendingCount').textContent = tasks.length - completed;
    }

    function renderTasks() {
      const list = document.getElementById('taskList');
      const empty = document.getElementById('emptyState');
      list.innerHTML = '';
      const visibleTasks = tasks.filter(task => currentFilter === 'all' || (currentFilter === 'done' ? task.done : !task.done));
      empty.hidden = visibleTasks.length > 0;
      empty.textContent = tasks.length === 0 ? 'No tasks yet. Add your first study task above.' : 'No tasks match this filter.';

      visibleTasks.forEach(task => {
        const item = document.createElement('li');
        item.className = `task${task.done ? ' done' : ''}`;
        const checkbox = document.createElement('input');
        checkbox.type = 'checkbox';
        checkbox.checked = task.done;
        checkbox.setAttribute('aria-label', `Mark ${task.text} as ${task.done ? 'pending' : 'completed'}`);
        checkbox.addEventListener('change', () => {
          task.done = checkbox.checked;
          saveTasks();
          renderTasks();
        });
        const label = document.createElement('span');
        label.className = 'task-text';
        label.textContent = task.text;
        const priority = document.createElement('span');
        priority.className = 'priority';
        priority.textContent = task.priority;
        const remove = document.createElement('button');
        remove.type = 'button';
        remove.className = 'delete';
        remove.textContent = 'Delete';
        remove.setAttribute('aria-label', `Delete ${task.text}`);
        remove.addEventListener('click', () => {
          tasks = tasks.filter(entry => entry.id !== task.id);
          saveTasks();
          renderTasks();
        });
        item.append(checkbox, label, priority, remove);
        list.appendChild(item);
      });
      updateStats();
    }

    document.getElementById('taskForm').addEventListener('submit', event => {
      event.preventDefault();
      const input = document.getElementById('taskInput');
      const text = input.value.trim();
      if (!text) return;
      tasks.unshift({ id: `${Date.now()}-${Math.random().toString(16).slice(2)}`, text, priority:
document.getElementById('priorityInput').value, done: false });
      saveTasks();
      input.value = '';
      input.focus();
      currentFilter = 'all';
      document.querySelectorAll('[data-filter]').forEach(button => button.classList.toggle('active', button.dataset.filter ===
'all'));
      renderTasks();
    });

    document.querySelectorAll('[data-filter]').forEach(button => {
      button.addEventListener('click', () => {
        currentFilter = button.dataset.filter;
        document.querySelectorAll('[data-filter]').forEach(entry => entry.classList.toggle('active', entry === button));
        renderTasks();
      });
    });

    renderTasks();
  </script>
</body>
</html>
```

7. Testing and Results

The application was opened in a Chromium-based browser and its main operations were tested. The results are summarized below:

Test case	Expected result	Status
Initial page load	Dashboard and sample tasks display correctly	**Passed**
Add a task	New task appears and counters update	**Passed**
Complete a task	Task is marked completed and counted	**Passed**
Completed filter	Only completed tasks are displayed	**Passed**
Delete a task	Selected task is removed	**Passed**
Browser storage	Task data persists through page refresh	**Passed**

Result: The tested functions operated as expected in the test browser. Browser localStorage is specific to the browser and device where the application is used; it is not an online account or cloud database.

8. GitHub Publishing Instructions

A personal GitHub repository must be created under the student's own account before a real repository URL can be added. This document therefore includes the official GitHub entry point and the exact steps; it does not claim that a repository has already been uploaded.

GitHub website: https://github.com/

GitHub guide for creating a repository: Creating a new repository

1. Sign in to your GitHub account and select New repository.
2. Enter the repository name: student-task-manager.
3. Choose Public if your instructor needs to view it, then create the repository.
4. Upload index.html and README.md (and optionally the assets folder) using Add file → Upload files.
5. Commit the uploaded files. Open the repository page and copy its URL.
6. Replace the placeholder below with that real repository URL before submitting the report.

Student repository URL: **[Paste your actual GitHub repository URL here after uploading]**

The source-code project folder contains index.html and README.md. The four screenshots are included in this report as evidence of application functionality.

9. Conclusion

The Student Task Manager demonstrates the development of a small but functional web application using core front-end technologies. It provides task management, priority selection, completion tracking, filtering, deletion, and local browser storage. The project meets the practical objective of building and testing an application, and it can be extended in the future with user accounts, cloud synchronization, due dates, and database support.